

#### 4-ma'ruza. Polimorfizm

##### Reja:

1. Virtual metodlar va ularning ustunligi
2. Yangi a'zolar bilan asosiy sinf a'zolarini yashirish
3. Statik va va dinamik polimorfizm

Obyektga yo'naltirilgan dasturlashda "Polimorfizm" tushunchasi inkapsulyatsiya va vorislikdan keyin uchinchi asosiy ustun hisoblanadi. Polimorfizm yunoncha so'z bo'lib, "ko'p shaklli" degan ma'noni anglatadi va uning ikki jihati bor:

- Kompilyatsiya vaqti polimorfizmi - ish vaqtida hosil bo'lgan sinf obyektlarining metod parametrlari tayanch sinf obyekti sifatida ko'rib chiqiladi. Ushbu polimorfizm paydo bo'lganda, obyekt e'lon qilingan turi uning ish vaqti turiga o'xshash bo'lmaydi.
- Ish vaqti polimorfizmi - tayanch sinflar virtual metodlarni belgilash, amalga oshirish hamda hosil bo'lgan sinflar ularni bekor qilishi mumkin.

##### Virtual metodlar va ularning ustunligi

Agar metod tayanch sinfda virtual deb e'lon qilingan bo'lsa, metod virtual deb ataladi. Virtual metod bir yoki bir nechta voris sinflarda bekor qilinishi mumkin. Shuning uchun har bir voris sinf virtual metodga ega bo'lishi mumkin. Bunday holda, havola orqali kirish mumkin bo'lgan obyekt turiga qarab chaqirilishi kerak bo'lgan virtual metodni aniqlaydi va bu ish vaqtida amalga oshiriladi. Shuning uchun har xil turdagi obyektlarga murojaat qilganda, virtual metodning turli xil versiyalari bajariladi. Boshqacha qilib aytganda, bajariladigan virtual metodning varianti ushbu obyektga havola turiga qarab emas, balki obyekt turiga qarab tanlanadi. Shunday qilib, agar tayanch sinf virtual metodni o'z ichiga olgan bo'lsa va undan voris sinf olinsa, u holda tayanch sinfga murojaat qilish orqali har xil turdagi obyektlarga murojaat qilganda, ushbu virtual metodning turli xil versiyalari bajariladi.

Metod o'z nomidan oldin virtual kalit so'z yordamida tayanch sinfda virtual deb e'lon qilinadi. Voris sinfda virtual metod bekor qilinganda *override* modifikatori ishlatiladi va voris sinfda virtual metodni qayta aniqlash jarayoni metodni qayta aniqlash deb ataladi. Bekor qilinganda *override* metodining nomi, turi virtual metod bilan aynan bir xil bo'lishi kerak. Bundan tashqari, virtual metodni *static* yoki *abstrakt* deb e'lon qilish mumkin emas.

Quyida virtual metodlar va ularning bekor qilinishini ko'rsatadigan misol keltirilgan.

```
using System;
class Tayanch
{
    // Tayanch sinfda virtual metod yaratish
    public virtual void Funksiya()
    {
        Console.WriteLine("Tayanch sinfidagi Funksiya() metodi");
    }
}
class Voris1 : Tayanch
{
    // Tayanch sinfdagi Funksiya() metodini bekor qilish
    public override void Funksiya()
    {
        Console.WriteLine("Voris1 sinfidagi Funksiya() metodi");
    }
}
class Voris2 : Tayanch
{
    // Voris2 sinfda Funksiya() metodini qayta belgilash
    public override void Funksiya()
```

```

    {
        Console.WriteLine("Vorisi2 sinfidagi Funksiya() metodi");
    }
}
class OverrideDemo
{
    static void Main()
    {
        Tayanch baza = new Tayanch();
        Vorisi1 d1 = new Vorisi1();
        Vorisi2 d2 = new Vorisi2();
        baza.Funksiya();
        baza = d1;
        baza.Funksiya();
        baza = d2;
        baza.Funksiya();
        Console.ReadKey();
    }
}

```

Kodni ishga tushirish natijasi

*Tayanch sinfidagi Funksiya() metodi*

*Vorisi1 sinfidagi Funksiya() metodi*

*Vorisi2 sinfidagi Funksiya() metodi*

Yuqoridagi misol Tayanch sinfini va undan vorisi olingan ikkita Vorisi1 va Vorisi2 sinfini yaratadi. Tayanch sinfi Funksiya() virtual metodini e'lon qiladi, bu ikkala vorisi sinfda ham bekor qilinadi. Main() metodida Tayanch, Vorisi1 va Vorisi2 turidagi obyektlar e'lon qilinadi. Tayanch mos havola turidagi baza o'zgaruvchisi e'lon qilinadi. Har bir obyekt turiga havola baza o'zgaruvchisiga tayinlanadi va keyin Funksiya() metodini chaqirish uchun ishlatiladi. Yuqoridagi kodning natijasidan ko'rinib turibdiki, bajarilayotgan Funksiya() metodining versiyasi baza o'zgaruvchisining sinf turiga qarab emas, balki ushbu metodga havola qilingan obyekt turiga qarab aniqlanadi.

Lekin virtual metodni bekor qilish shart emas. Agar vorisi sinf virtual metodning versiyasini taqdim etmasa, unda quyidagi misolda bo'lgani kabi uning tayanch sinfidagi versiyasi ishlatiladi.

```

using System;
class Tayanch
{
    // Tayanch sinfda virtual metod yaratish
    public virtual void Funksiya()
    {
        Console.WriteLine("Tayanch sinfidagi Funksiya() metodi");
    }
}
class Vorisi1 : Tayanch
{
    // Vorisi sinfda Funksiya() metodini bekor qilish
    public override void Funksiya()
    {
        Console.WriteLine("Vorisi1 sinfidagi Funksiya() metodi");
    }
}
class Vorisi2 : Tayanch
{

```

```

// Voris2 voris sinfda Funksiya() metodini qayta belgilash
public override void Funksiya()
{
    // Bu sinfda Funksiya() metodi bekor qilinmaydi.
}
}

class OverrideDemo
{
    static void Main()
    {
        Tayanch baza = new Tayanch();
        Voris1 d1 = new Voris1();
        Voris2 d2 = new Voris2();
        Tayanch AsosiyRef; // tayanch sinfga havola
        AsosiyRef = baza;
        AsosiyRef.Funksiya();
        AsosiyRef = d1;
        AsosiyRef.Funksiya();
        AsosiyRef = d2;
        AsosiyRef.Funksiya(); // Tayanch sinfidan Funksiya() metodi chaqiriladi
        Console.ReadKey();
    }
}

```

Dastur natijasi:

*Tayanch sinfidagi Funksiya() metodi*

*Voris1 sinfidagi Funksiya() metodi*

Bu misolda Funksiya() metodi Voris2 sinfida bekor qilinmagan. Shuning uchun Voris2 sinfining obykti uchun Tayanch sinfidan Funksiya() metodi chaqiriladi. Agar ko'p darajali iyerarxiya mavjud bo'lsa, virtual metod voris sinfda bekor qilinmasa, quyidagi misolda bo'lgani kabi iyerarxiyaning eng yaqin varianti bajariladi.

```

using System;
class Tayanch
{
    // Tayanch sinfda virtual metod yaratish
    public virtual void Funksiya()
    {
        Console.WriteLine("Tayanch sinfidagi Funksiya() metodi");
    }
}

class Voris1 : Tayanch
{
    // voris sinfda Funksiya() metodini bekor qilish
    public override void Funksiya()
    {
        Console.WriteLine("Voris1 sinfidagi Funksiya() metodi");
    }
}

class Voris2 : Voris1
{
    // Bu sinfda Funksiya() metodi bekor qilinmaydi
}

```

```

class Voris3 : Voris2
{
    // Bu sinfda Funksiya() metodi bekor qilinmaydi
}
class NoOverrideDemo2
{
    static void Main()
    {
        Voris3 d = new Voris3();
        Tayanch AsosiyRef; // Tayanch sinfiga havola
        AsosiyRef = d;
        AsosiyRef.Funksiya(); // Voris1 sinfidan Funksiya() metodini chaqirish
        Console.ReadKey();
    }
}

```

Dastur natijasi

*Voris1 sinfidagi Funksiya() metodi*

Ushbu misolda Voris3 sinfi Voris2 sinfini, Voris2 sinfi esa Voris1 sinfini, Voris1 sinfi navbatida Tayanch sinfini voris qilib oladi. Yuqoridagi natijadan ko‘rinib turibdiki, Voris1 da bekor qilingan Funksiya() metodi bajariladi, chunki u Voris3 va Voris2 iyerarxiyasidan Funksiya() metodi bekor qilinmagan Voris1 ga o‘tishda aniqlangan birinchi virtual metoddir.

**Eslatma:** xossalarni *virtual* kalit so‘zi bilan o‘zgartirish va *override* kalit so‘zi bilan bekor qilish mumkin.

### Yangi a‘zolar bilan tayanch sinf a‘zolarini yashirish

Agar voris sinfda tayanch sinf a‘zosi bilan bir xil nomdagi a‘zo bo‘lishi kerak bo‘lsa, *new* kalit so‘z bilan tayanch sinf a‘zosini yashirish mumkin.

Quyidagi dastur kodi misol sifatida keltiradi:

```

public class TayanchClass
{
    public void Metod()
    {
        WorkField++;
    }
    public int WorkField;
    public int WorkProperty
    {
        get { return 0; }
    }
}
public class VorisClass : TayanchClass
{
    public new void Metod() { WorkField++; }
    public new int WorkField;
    public new int WorkProperty
    {
        get { return 0; }
    }
}

```

**Eslatma:** *Virtual* a‘zolar ushbu dasturning tayanch sinfini chaqirish uchun foydalanishi tavsiya etiladi.

Polimorfizm statik yoki dinamik bo‘lishi mumkin. Statik polimorfizmda metodga javob kompilyatsiya vaqtida aniqlanadi. Dinamik polimorfizm u ish vaqtida hal qilinadi.

## Statik polimorfizm

Kompilyatsiya vaqtida metodni obyekt bilan bog'lash mexanizmi statik bog'lash deb ataladi.

C# statik polimorfizmni amalga oshirishning ikkita usulini taqdim etadi.

1. Qayta yuklash metodlari;
2. Operatorning qayta yuklanishi.

### Qayta yuklash metodlari

Bir xil metod nomi uchun har xil doirada bir nechta metod ta'riflariga ega bo'lishi mumkin. Metodlarni qayta yuklash - bir-biridan turlari va yoki argumentlar ro'yxatidagi argumentlar soni bo'yicha farq qilishi kerak. Faqat qaytarish turidan farq qiladigan metod e'lonlarini qayta yuklab bo'lmaydi.

Quyidagi misol har xil turdagi ma'lumotlarni chop etish uchun print() metodidan foydalanilish keltirilgan:

```
using System;
namespace PolymorphismApplication {
    class Printdata {
        void print(int i) {
            Console.WriteLine("int turida chop etish: {0}", i);
        }
        void print(double f) {
            Console.WriteLine("float turida chop etish: {0}", f);
        }
        void print(string s) {
            Console.WriteLine("string turida chop etish: {0}", s);
        }
        static void Main(string[] args) {
            Printdata p = new Printdata();
            // Butun sonni chop etish uchun print() ga murojaat qilish
            p.print(5);
            // Haqiqiy sonni chop etish uchun print() ga murojaat qilish
            p.print(500.263);
            // Satrni chop etish uchun print() ga murojaat qilish
            p.print("Hello C#");
            Console.ReadKey();
        }
    }
}
```

Yuqoridagi kod kompilyatsiya qilingan va bajarilganida, u quyidagi natijani beradi:

**int** turida chop etish: 5

**float** turida chop etish: 500.263

**string** turida chop etish: Hello C#

## Dinamik polimorfizm

C# interfeys sinfining qisman bajarilishini ta'minlash uchun ishlatiladigan abstrakt sinflarni yaratishga imkon beradi. Abstrakt sinflar voris sinf tomonidan amalga oshiriladigan abstrakt metodlarni o'z ichiga oladi.

*Abstrakt sinfdan tashqarida abstrakt metodni e'lon qilib bo'lmaydi. Sinf override deb e'lon qilinganda, undan voris olinmaydi, abstrakt sinflar override deb e'lon qilinishi mumkin emas.*

Virtual metodlar turli xil voris sinflarda turli xil metodlar bilan amalga oshirilishi mumkin va ushbu metodlarga chaqirish ish vaqtida hal qilinadi. Dinamik polimorfizm abstrakt sinflar va virtual metodlar bilan amalga oshiriladi.

Quyidagi dastur buni namoyish etadi:

```

using System;
namespace PolymorphismApplication {
    class Shakl {
        protected int width, height;
        public Shakl( int a = 0, int b = 0) {
            width = a;
            height = b;
        }
        public virtual int area() {
            Console.WriteLine("Tayanch sinf a'zosi: ");
            return 0;
        }
    }
    class Turtburchak: Shakl {
        public Turtburchak( int a = 0, int b = 0): base(a, b) {
        }
        public override int area () {
            Console.WriteLine("Turtburchak sinf a'zosi: ");
            return (width * height);
        }
    }
    class Uchburchak: Shakl {
        public Uchburchak(int a = 0, int b = 0): base(a, b) {
        }
        public override int area() {
            Console.WriteLine("Uchburchak sinf a'zosi: ");
            return (width * height / 2);
        }
    }
    class Caller {
        public void CallArea(Shakl sh) {
            int a;
            a = sh.area();
            Console.WriteLine("Maydon: {0}", a);
        }
    }
    class Tester {
        static void Main(string[] args) {
            Caller c = new Caller();
            Turtburchak r = new Turtburchak(10, 7);
            Uchburchak t = new Uchburchak(10, 5);
            c.CallArea(r);
            c.CallArea(t);
            Console.ReadKey();
        }
    }
}

```

Yuqoridagi kod kompilyatsiya qilinganda va bajarilganda, quyidagi natijani beradi:

*Turtburchak sinf a'zosi:*

*Maydon: 70*

*Uchburchak sinf a'zosi:*

*Maydon: 25*

